

NFA 与 DFA 的对比

以正则表达式 $(a|b)^*abb$ 为例

1 非确定有限自动机 (NFA)

NFA 的核心特征

- 非确定性：同一状态下，读入同一字符可以转移到多个不同状态
- ϵ 转换：可以不消耗任何输入字符就发生状态转移（本图中用紫色虚线表示）
- 更紧凑：通常状态数更少，结构更直观地反映正则表达式

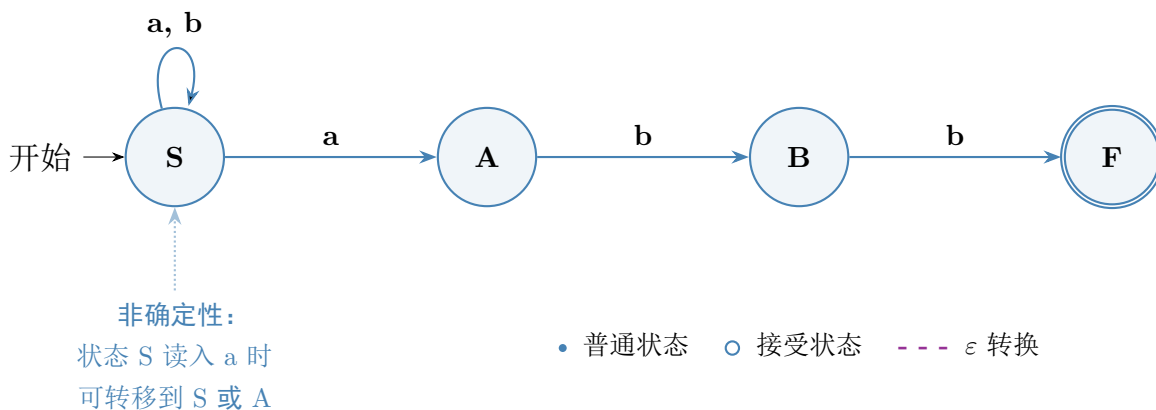


图 1: NFA 识别 $(a|b)^*abb$ (Thompson 构造法简化版)

NFA 的工作方式

状态 S 是核心——它对应 $(a|b)^*$ 部分：

- 在 S 上遇到 a 或 b，可以”原地踏步”（继续匹配 $(a|b)^*$ ）
- 在 S 上遇到 a，也可以”猜测”这可能是 abb 的开头，转移到 A
- 这就是非确定性：同一输入 a 有两条出路！

2 确定有限自动机 (DFA)

DFA 的核心特征

- 确定性：每个状态对每个输入符号，恰好有一条 转移边
- 无 ϵ 转换：所有转移都消耗输入字符
- 状态更多：通过”子集构造法”从 NFA 转换而来，状态对应 NFA 状态的集合

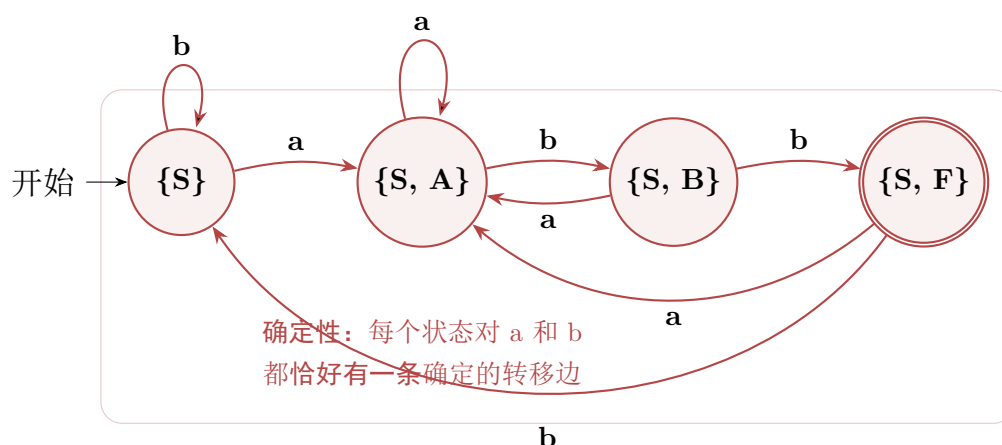


图 2: DFA 识别 $(a|b)^*abb$ (NFA 经子集构造法得出)

DFA 的状态含义

每个 DFA 状态是 NFA 状态的集合，记录了”当前可能处于哪些 NFA 状态”：

DFA 状态	包含的 NFA 状态	含义
{S}	S	还没有匹配到 abb 的任何前缀
{S, A}	S, A	最后一个字符是 a (可能是 abb 的开头)
{S, B}	S, B	最后两个字符是 ab
{S, F}	S, F	已经匹配到 abb! (接受状态)

3 NFA vs DFA ——直观对比

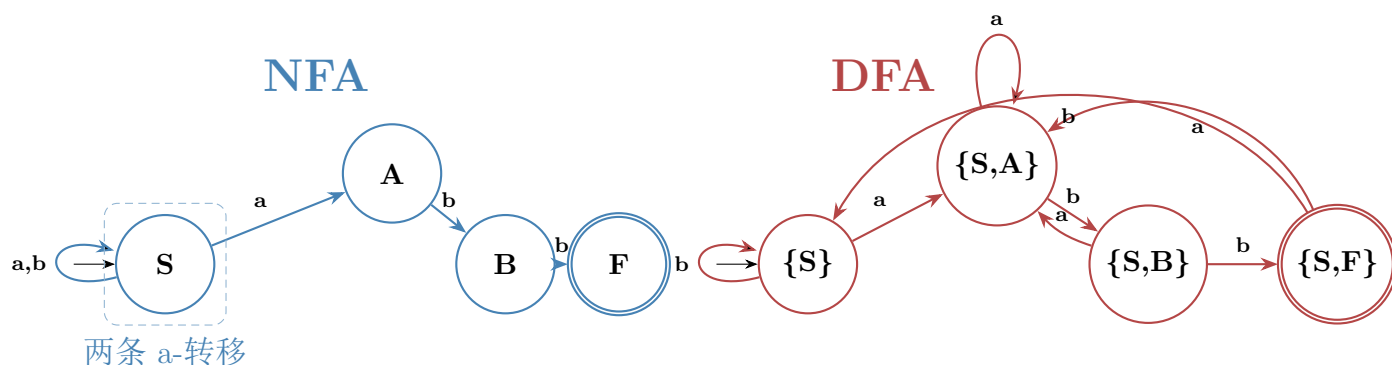


图 3: NFA 与 DFA 并排对比

核心结论

1. NFA 和 DFA 表达能力等价——它们都恰好识别正则语言
2. NFA 更紧凑，便于从正则表达式构造；DFA 更高效，便于程序实现
3. 实际编译器中的词法分析器：先用正则表达式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ 最小化 DFA，最终用 DFA 驱动
4. 本例中 $(a|b)^*abb$ 恰好 NFA 和 DFA 状态数相同（4 个），但通常 DFA 状态数可能指数级膨胀

对比维度	NFA	DFA
状态数	4 个（较少）	4 个（本例恰好相同）
转移确定性	不确定：	
S 读入 a 可到 {S, A}	确定：	
每个状态对 a/b 均唯一转移		
ϵ 转换	可有（紧凑版）	绝无
模拟/实现难度	需回溯或并行模拟	直接模拟， $O(n)$ 时间
构造难度	容易：从正则表达式	
直接构造（Thompson 法）	较难：需经 $NFA \rightarrow DFA$	
子集构造法		
与正则表达式对应	直观，结构一一对应	不直观

表 1: NFA 与 DFA 详细对比